

Melody Plagiarism Detector - MPEI Practical Work

Inês Batista, 124877 & Maria Quinteiro, 124996

1. Introdução

O presente relatório descreve o desenvolvimento e o funcionamento do projeto Music Plagiarism Detector, uma ferramenta desenvolvida em MATLAB com o objetivo de identificar possíveis casos de plágio musical através da comparação automática de melodias representadas em formato MIDI. A ideia central do projeto consiste em transformar cada ficheiro MIDI numa sequência compacta que capture o movimento melódico e o ritmo da peça, e depois medir o grau de semelhança entre duas dessas sequências. Para isso recorremos a técnicas de comparação de conjuntos, nomeadamente k-shingles e a semelhança de Jaccard, complementadas por estruturas de dados eficientes, o Filtro de Bloom, o MinHash com LSH, e por um classificador probabilístico de Naive Bayes para cenários em que existem dados rotulados disponíveis. O sistema considera que existe necessidade de atenção do utilizador face à possibilidade de plágio quando o valor de semelhança de Jaccard calculado entre dois ficheiros está entre 0.01 e 0.03, valor acima destes são bastante suspeitos. Este limiar foi definido de forma conservadora para capturar qualquer sobreposição mensurável de padrões melódicos, incluindo casos de plágio parcial. Na prática, dois ficheiros completamente distintos tendem a obter valores muito próximos de zero, pelo que qualquer resultado acima de 0,01 é digno de análise.

O projeto está organizado em três blocos principais: a extração e normalização de eventos MIDI (implementada em `midi_to_note_events.m`), a conversão para sequências de (intervalo melódico, duração quantizada) (`melody_events_to_interval_duration_sequence.m`), e o motor de comparação e deteção (`detection.m`, `main.m`). Existe ainda um módulo dedicado à geração de relatórios sobre o dataset (`generate_dataset_report.m`) e uma suíte de testes que valida cada componente individualmente e o pipeline integrado.

2. O Dataset MIDI (MAESTRO)

Para testar e validar o sistema foi utilizado o dataset MAESTRO (MIDI and Audio Edited for Synchronous TRacks and Organization), um conjunto de dados público desenvolvido pela Google com gravações de piano clássico de alto nível de execução. O MAESTRO contém centenas de ficheiros MIDI correspondentes a interpretações reais de pianistas em competições internacionais, tornando-o um recurso ideal para testar sistemas de análise musical automática. Cada ficheiro MIDI do dataset representa uma interpretação completa de uma peça de piano, com toda a informação de notas, durações, velocidades e timings preservada com elevada precisão. O dataset está organizado por ano de gravação e é colocado na pasta `data/maestro-v3.0.0/` dentro da raiz do projeto para ser reconhecido automaticamente pelas funções.

Um ficheiro MIDI (Musical Instrument Digital Interface) não é um ficheiro de áudio mas sim uma sequência de instruções musicais. Esta estrutura torna os ficheiros MIDI muito compactos e, sobretudo, diretamente interpretáveis por computador sem necessidade de reconhecimento de áudio. Um ficheiro MIDI é composto por um cabeçalho (MThd) que define o

formato do ficheiro e o número de ticks por semínima (*ticks_per_quarter*), seguido de uma ou mais faixas (MTrk). Cada faixa contém uma sequência de eventos separados por tempos delta, que representam o intervalo de tempo entre um evento e o anterior. Os eventos mais importantes para este projeto são os eventos *note on* (nota começa) e *note off* (nota termina). Para cada par *note on*/*note off* da mesma nota e canal é possível calcular o *onset* (quando começou) e a duração da nota. O timing nos ficheiros MIDI é expresso em *ticks*, uma unidade de tempo interna ao ficheiro. Para converter *ticks* em *beats* divide-se o valor em *ticks* pelo campo *ticks_per_quarter* do cabeçalho. Esta conversão é importante porque garante que a representação final em *beats* é independente do andamento da peça, tornando a comparação invariante à velocidade de execução.

3. Extração e Normalização de eventos MIDI

A função `midi_to_note_events.m` é responsável por ler um ficheiro MIDI e transformar os seus eventos brutos numa representação estruturada e normalizada. **O processo é feito lendo o ficheiro byte a byte em modo *big-endian*, começando pela leitura do cabeçalho MThd, onde são extraídos o formato do ficheiro (0, 1 ou 2), o número de faixas e o valor *ticks_per_quarter*. De seguida, cada faixa MTrk é lida sequencialmente.** Os eventos de tempo delta são codificados em MIDI através de um formato de comprimento variável (*variable-length quantity*), onde o bit mais significativo de cada byte indica se existe mais um byte a seguir. A função auxiliar `read_varlen` trata desta descodificação, acumulando o valor final em base 128 através de um ciclo que lê bytes enquanto o bit mais significativo estiver ativo. Para cada evento na faixa, a função identifica o tipo de mensagem a partir do byte de *status*. Os eventos *note on* com velocidade positiva são registados como início de nota no array `active`, indexado por canal e número de nota. Os eventos *note off* (ou *note on* com velocidade zero, que o standard MIDI trata como *note off*) fecham a nota correspondente através da função `close_note`, calculando a duração como a diferença entre o *tick* de fecho e o *tick* de início. Eventos de outro tipo - mudanças de controlo, *aftertouch*, *pitch bend*, eventos meta - são ignorados pois não são relevantes para a comparação melódica. O resultado é um array de estruturas em que cada elemento representa uma nota com os campos: `note_number` (número MIDI de 0 a 127), `pitch_class` (classe de altura de 0 a 11, resultado da operação módulo 12 sobre `note_number`), `note_name` (nome textual como "A4" obtido por mapeamento de altura e oitava), `onset_tick`, `duration_tick`, `onset_beat` (obtido pela divisão de `onset_tick` por `ticks_per_quarter`), `duration_beat` (obtido de forma análoga), `velocity`, `off_velocity`, `channel` e `track`. No final, os eventos são ordenados por `onset_tick` crescente, e em caso de empate a nota mais aguda surge primeiro. Esta ordenação é crítica para os passos subsequentes, pois garante que a sequência melódica respeita a ordem temporal das notas. Para gerir o *running status* do MIDI (um mecanismo de compressão onde o byte de *status* pode ser omitido em eventos consecutivos do mesmo tipo), a função mantém o último *status* lido e, quando encontra um byte com o bit mais significativo a zero, reutiliza esse *status* em vez de ler um novo.

4. Conversão para sequência de intervalo e duração

A função `melody_events_to_interval_duration_sequence.m` converte os eventos normalizados numa sequência de pares (intervalo melódico, duração quantizada). Para estes ficheiros que estamos a usar como dataset, vários sons podem começar simultaneamente. A

estratégia por defeito, chamada *top_note_per_onset*, seleciona apenas a nota mais aguda em cada instante de início. O algoritmo percorre todos os *onsets* únicos presentes nos eventos, identifica todas as notas que começam nesse instante, e escolhe aquela com o maior valor de *note_number*. Se existir empate de altura, a nota com maior duração é preferida, determinada pela comparação do campo *duration_beat*. Esta abordagem, embora simples, funciona bem para música para piano onde a melodia está frequentemente na voz superior. As durações são quantizadas para a grelha rítmica mais próxima definida pelo parâmetro *duration_grid*, cujo valor por defeito é 0,25 *beats*, equivalente a uma semicolcheia. A quantização é calculada dividindo a duração em *beats* pelo valor da grelha, arredondando ao inteiro mais próximo, e multiplicando novamente pelo valor da grelha. Esta operação pode ser expressa como **$\text{duration_quantized} = \text{round}(\text{duration_beat} / \text{duration_grid}) * \text{duration_grid}$** . A quantização torna o sistema robusto a pequenas variações rítmicas que não afetam a identidade musical da peça. Os intervalos melódicos são calculados como a diferença entre números MIDI consecutivos, preservando o sinal (positivo para movimento ascendente, negativo para descendente). Esta representação é invariante à transposição porque o intervalo entre duas notas não depende da tonalidade absoluta da peça; uma melodia transportada para uma tonalidade diferente mantém exatamente os mesmos intervalos entre notas consecutivas. O **resultado final é uma matriz Nx2 onde cada linha contém um par [intervalo_melódico, duração_quantizada]**.

5. Geração de Shingles e Funções de Hash

Antes de descrever o terceiro módulo, é necessário compreender como as sequências melódicas são transformadas em conjuntos de números inteiros comparáveis - os *shingles* - e qual o papel das funções de *hash* nesse processo. Um **k-shingle** é uma janela deslizante de *k* elementos consecutivos sobre uma sequência. Com uma sequência de *N* pares e janela de tamanho *k*, são produzidas $N - k + 1$ janelas sobrepostas. Esta sobreposição é deliberada: dois fragmentos que diferem numa nota no meio ainda partilham vários shingles, o que faz com que a semelhança calculada seja proporcional ao grau real de sobreposição melódica, ao contrário de uma abordagem baseada em blocos não sobrepostos que perderia essa sensibilidade. Cada janela de *k* pares é primeiro serializada numa lista plana de números inteiros, intercalando intervalos e durações.

As durações, que são valores fracionários em *beats*, são multiplicadas por mil e arredondadas para inteiro antes da serialização, evitando problemas de representação em vírgula flutuante que poderiam fazer com que durações matematicamente iguais mas computacionalmente diferentes produzissem *hashes* distintos. Sobre este vetor é aplicada uma *hash* polinomial de rolamento (*polynomial rolling hash*), que começa com um acumulador a zero e, para cada elemento do vetor, multiplica o acumulador por uma base (número primo 131) e adiciona o elemento mais uma constante de deslocamento (1000003), reduzindo o resultado pelo módulo de um número primo de Mersenne, nomeadamente dois elevado a trinta e um menos um.

A constante de deslocamento serve para afastar os valores de zero, evitando que sequências com zeros iniciais produzam *hashes* idênticos. A base 131 é escolhida por ser um número primo ligeiramente acima de 128, o que é suficiente para distinguir qualquer par de valores inteiros numa janela melódica típica. O resultado final é um único inteiro que representa toda a janela de *k* pares melódicos. **Duas janelas idênticas produzem sempre o mesmo hash; duas janelas diferentes produzem com elevada probabilidade hashes diferentes.** O conjunto final de shingles de uma melodia é o conjunto dos *hashes* únicos de todas as janelas, obtido

através da função `unique`. Esta função de *hash* é usada na função `get_interval_duration_shingle_set.m`, que é o módulo principal de geração de shingles.

6. Comparação e Deteção

O terceiro módulo integra três componentes principais: o Filtro de Bloom, o MinHash com LSH e o classificador Naive Bayes, orquestrados pelas funções `detection.m` e `main.m`. O **Filtro de Bloom** é uma estrutura de dados probabilística que permite verificar a pertença de um elemento a um conjunto de forma muito eficiente. Internamente, é um array de bits inicialmente todos a zero. Para inserir um shingle, calculam-se várias posições no array a partir do seu valor de *hash* e colocam-se esses bits a um. Para verificar se um shingle pertence ao conjunto, calculam-se as mesmas posições e verifica-se se todos os bits estão a um. **Se algum bit estiver a zero, o shingle certamente não foi inserido. Se todos estiverem a um, o shingle provavelmente foi inserido, mas pode ser um falso positivo.** A grande vantagem do Filtro de Bloom é que nunca produz falsos negativos e ocupa pouca memória. No contexto do projeto, a função `bloom_count_matches` conta quantos shingles de uma consulta passam no teste contra o filtro de uma melodia da base de dados. O rácio entre essa contagem e o número total de shingles da consulta é usado como pré-filtro: apenas melodias com rácio acima de 0,5 são consideradas candidatas para verificação exata. O **MinHash com LSH** resolve o problema da comparação exaustiva entre todas as melodias. Em vez de comparar todos os pares possíveis, o que seria inviável para *datasets* grandes, o MinHash comprime cada conjunto de shingles numa assinatura compacta de cem números inteiros. Cada posição da assinatura é obtida aplicando uma função de *hash* diferente a todos os shingles do conjunto e guardando o valor mínimo. A propriedade fundamental é que a probabilidade de duas assinaturas coincidirem numa dada posição é exatamente igual à semelhança de Jaccard entre os conjuntos originais. O LSH (Locality Sensitive Hashing) divide estas assinaturas em vinte bandas de cinco posições cada. Para cada banda, calcula-se um *hash* da sub-assinatura. **Melodias que produzem o mesmo *hash* numa mesma banda são colocadas no mesmo *bucket* e consideradas candidatas a similares. A lógica é que melodias muito semelhantes têm alta probabilidade de coincidir em pelo menos uma banda, enquanto melodias diferentes têm probabilidade muito baixa.** A função `find_similar_melodies_lsh` implementa este processo e retorna apenas os pares de melodias que partilham pelo menos uma banda, calculando depois a semelhança de Jaccard exata apenas para esses candidatos.

O classificador Naive Bayes é utilizado quando existem melodias rotuladas como "plagiadas" ou "originais", permitindo classificar novas melodias com base nos padrões de shingles aprendidos durante o treino. A versão implementada é o Multinomial Naive Bayes, adequada para *features* que representam contagens, neste caso, quantas vezes cada shingle aparece numa melodia. O classificador baseia-se no teorema de Bayes com a assunção simplificadora de que todas as *features* são independentes entre si dada a classe. **Trabalha com logaritmos para evitar problemas de *underflow* numérico e aplica *smoothing* de Laplace com parâmetro igual a um para evitar que shingles não vistos durante o treino anulem a probabilidade de uma classe.** A função principal `naive_bayes` aceita tanto matrizes numéricas como *cell arrays* de shingles, constrói automaticamente o vocabulário a partir dos dados de treino, e retorna as classes previstas para os dados de teste.

O programa principal `main.m` orquestra todo o pipeline, oferecendo vários modos de execução. O modo de demonstração, ativado com `d = main('demo_mode', true);`, executa quatro cenários controlados que ilustram o comportamento esperado do sistema: auto-comparação com semelhança esperada de um, comparação contra melodias distintas com semelhança

baixa, construção de shingles híbridos com cinquenta por cento de cada melodia resultando em semelhança próxima de 0,5, e um mini *all-vs-all* sobre um subconjunto configurável. O modo *all-vs-all* completo é executado com `r = main();`, sendo possível limitar o número de ficheiros processados com `r = main('max_database', 20);`. A função `generate_dataset_report()` produz um relatório completo sobre o dataset, incluindo um ficheiro de texto com estatísticas gerais, um ficheiro CSV com os pares mais similares, um histograma da distribuição de semelhanças e um gráfico de *ranking* dos candidatos para a consulta mais representativa.

7. Testes e Validação

O projeto inclui uma suíte de testes abrangente, organizada na pasta `tests/`, que cobre todos os componentes individuais e o pipeline integrado. Os testes podem ser executados com o comando `run('tests/test.m');`, que chama sequencialmente os testes de cada módulo. O teste do Filtro de Bloom verifica que não existem falsos negativos e que o contador de correspondências funciona corretamente. O teste do MinHash e LSH confirma que dois conjuntos idênticos produzem assinaturas iguais e que o LSH identifica corretamente pares similares mesmo na presença de uma terceira melodia completamente diferente. O teste do Naive Bayes valida o classificador tanto em modo numérico como em modo de shingles, com o *smoothing* de Laplace a garantir estabilidade numérica. O teste de integração do dataset confirma que o pipeline completo funciona com ficheiros MIDI reais do MAESTRO, produzindo sequências Nx2 consistentes e candidatos com campos de semelhança corretamente preenchidos.

Importa notar que a execução dos testes sobre o dataset MAESTRO completo é demorada, uma vez que o sistema processa cada ficheiro como consulta e compara com todos os restantes. Para um dataset de mil melodias, o número de comparações é da ordem das dezenas de milhares, e cada comparação envolve a extração da sequência melódica, a geração de shingles, o pré-filtro com o Filtro de Bloom e, para os candidatos que ultrapassam o limiar, o cálculo exato da semelhança de Jaccard. Por este motivo, durante o desenvolvimento e a validação inicial, recomenda-se utilizar um subconjunto reduzido do dataset, por exemplo com o parâmetro `max_database` igual a vinte ou trinta ficheiros, para obter resultados em tempo útil. O modo de demonstração com `demo_all_vs_all_size` igual a quatro ou oito ficheiros é ainda mais rápido e suficiente para verificar o comportamento correto do sistema antes de uma corrida completa.

8. Conclusão

O projeto apresenta uma pipeline funcional para deteção de semelhança melódica em ficheiros MIDI. A combinação de uma representação compacta baseada em intervalos e durações com estruturas de comparação eficientes resulta num sistema capaz de analisar grandes bases de dados musicais de forma escalável. O critério de plágio adotado, semelhança de Jaccard superior a 0.01, é conservador e garante que qualquer sobreposição mensurável é sinalizada para posterior análise humana. A suíte de testes abrangente garante a correção do sistema em vários cenários e facilita a manutenção futura do código, enquanto a arquitetura modular permite a substituição ou melhoria de componentes individuais sem afetar o restante do sistema. As principais limitações residem na extração melódica para ficheiros polifónicos onde a melodia principal não é a voz mais aguda, e na perda de informação de ordem global inerente à abordagem baseada em conjuntos de shingles.